



Greedy Attack: Breaking Finality against VeChain Proof-of-Authority Consensus Protocol

Rujia Li
Tsinghua University
State Key Laboratory of Cryptography
and Digital Economy Security
Beijing, China
rujia@tsinghua.edu.cn

Qin Wang
CSIRO Data61
Sydney, Australia
qinwangtech@gmail.com

Haochen Wang
The Hong Kong University of Science
and Technology
Hong Kong, China
whc@ust.hk

Xueqian Lu
Independent Researcher
Beijing, China
xueqian.lu@bitheart.org

Sisi Duan✉†
Tsinghua University
State Key Laboratory of Cryptography
and Digital Economy Security
Beijing, China
duansisi@tsinghua.edu.cn

Abstract

Finality gadget is a critical component in modern blockchain systems. Ever since Ethereum used the Casper finality gadget, many variants of finality approaches have been adopted to make it possible to finalize blocks, i.e., when a block is finalized, its order will never be reversed. In this work, we study the finality approach by VeChainThor, the consensus protocol of VeChain blockchain. We propose *greedy attack*, an attack in which a block proposed by an honest sealer can be finalized with a probability of only 0.19%. We validate our results via theoretical analysis and extensive experimentation. Our findings have been disclosed and acknowledged by the VeChain team, and we are currently working with them to improve the VeChainThor protocol.

CCS Concepts

• Security and privacy → Distributed systems security.

Keywords

Consensus Protocol; Finality Attack; Blockchain

ACM Reference Format:

Rujia Li, Qin Wang, Haochen Wang, Xueqian Lu, and Sisi Duan✉. 2026. Greedy Attack: Breaking Finality against VeChain Proof-of-Authority Consensus Protocol. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3774904.3792513>

Resource Availability:

*Corresponding author.

†Sisi is also with National Financial Cryptography Research Center, Zhongguancun Laboratory, and Shandong Institute of Blockchains.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates.*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2307-0/2026/04

<https://doi.org/10.1145/3774904.3792513>

The source code and data of this paper have been made publicly available at <https://doi.org/10.5281/zenodo.18344489>.

1 Introduction

Finality in blockchains denotes the fact that once a block is finalized, its order will never be reversed. Finality of blocks is deterministic in conventional Byzantine fault-tolerant (BFT) protocols [2, 9, 15, 46], but probabilistic in systems such as Bitcoin [20, 38] and Ethereum 1.0 [45]. When Ethereum transitioned to Proof-of-Stake (PoS) in its 2.0 version, it borrowed the concept of finality from conventional BFT and proposed the Friendly Finality Gadget (FFG) [6, 7]. Finality gadget has now become a fundamental tool in many blockchain systems, and many variants are developed, such as the fast finality mechanism in BNB Smart Chain [23] and the GRANDPA mechanism in Polkadot [36].

VeChain and its consensus protocol VeChainThor. VeChain [39], established in 2017, is now ranked 46th in the cryptocurrency market. It has a market capitalization exceeding \$2 billion USD and a user base of more than 1.3 million. VeChain employs a variant of the Proof-of-Authority (PoA) Aura protocol [44] called *VeChainThor* [40–42] as its consensus algorithm. VeChainThor follows the general design of PoA protocols. Briefly speaking, Aura operates among a set of sealers (also called validators, nodes, or replicas in the literature) where sealers propose blocks in a round-robin manner. On top of Aura, VeChainThor has made several optimizations. First, unlike Aura, where sealers take turns proposing blocks, sealers in VeChainThor are randomly selected via the Verifiable Random Function (VRF) [25]. Second, unlike Aura, which does not have a clear notion of *finality* [13], VeChainThor introduces its finality gadget called *Finality with One Bit* (FOB) [51]. FOB aims to achieve finality, i.e., if a block is finalized, its order will never be reversed. In this way, the system achieves *safety* [8].

Similar to FFG, FOB allows sealers to vote for blocks. However, it does not introduce a dedicated "vote" message. Instead, each sealer uses a bit in its block proposal to denote a vote, i.e., "1" means that the sealer extends its canonical chain, and "0" otherwise. Once $N - f$ sealers cast a "1" vote in their proposals for the same block

B , then B is *justified*, where N is the number of sealers and f is the maximum number of Byzantine sealers. If a block that extends B is justified, B is *finalized*.

An attack against the finality mechanism of VeChainThor. We present the *greedy attack*, an attack against the finality mechanism of VeChainThor. Under our attack, each block proposed by an honest sealer is expected to be finalized with a probability of only 0.19%! As a result, the FOB mechanism almost fails to finalize any blocks. The expected time to finalize a block is also significantly increased, making the performance even worse than that of Aura.

Our attack can be viewed as a variant of reorganization attack (reorg attack for short), a type of attack previously found in Proof-of-Work (PoW) [17], Proof-of-Stake (PoS) [11, 30, 32], and PoA [43, 49, 50]. In a reorg attack, Byzantine sealers withhold a private chain and monitor the public chain of blocks proposed by honest sealers. At a certain time, the private chain is released, after which blocks by honest sealers are reorganized and discarded. In our greedy attack, the Byzantine sealers repeatedly launch reorg attacks; it greedily chooses its strategy to *maximize* the number of reorganized blocks. Compared to a plain reorg attack, our greedy attack additionally manipulates the “bits” in the blocks to significantly lower the probability of finalizing the blocks. To our knowledge, greedy attack is the first to show that the finality mechanism of PoA can be violated via reorganization of the blocks (see Table 1 for a detailed comparison of greedy attack vs. known attacks).

Responsible disclosure. We have contacted the VeChain team and received confirmation on the effectiveness of our attacks, as reported in #Discussion.

Summary. We make the following contributions.

- We present a formal treatment of the VeChainThor protocol and its finality mechanism based on the documentation and codebase, mainly because VeChainThor is not published (§3).
- We present the *greedy attack* (the first known attack) against VeChainThor. Our attack shows that with overwhelming probability (≥ 99.81), the FOB mechanism fails to finalize any blocks from honest sealers (§4).
- We provide theoretical analysis and show that 28.57% of blocks (proposed by honest sealers) are expected to be reorganized, and each block proposed by an honest sealer is expected to be finalized with a probability of only 0.19% (§5).
- We validate the greedy attack using the open-source implementation of VeChainThor (§6). Our experimental results validate that only a small fraction of blocks can be finalized. All our actions and results were responsibly reported to the VeChain team.
- We analyze the choices of system parameters of VeChainThor to mitigate our attack (§7). Our analysis shows that the epoch (a time unit in VeChainThor) length need to be increased to at least 361 from 180. Otherwise, under the current configuration, the system can only tolerate 16% Byzantine sealers, instead of 33% as claimed.

2 System Model

Cryptographic assumptions. VeChainThor uses collision-resistant hash functions, denoted as $\text{Hash}()$, and digital signatures. The digital signature is assumed to be unforgeable.

Network assumption. VeChainThor does not explicitly define its timing assumptions. Based on the documentation [51], we believe that their timing assumption is a partially synchronous network [16], where there exists an unknown upper bound Δ for message processing and propagation. Since our attacks can be launched under arbitrary network conditions, we assume that the protocol executes in a synchronous network with a known upper bound Δ on message processing and propagation delays. We use the term “broadcast” to denote best effort broadcast [8], where the sender sends a message to all sealers.

Threat model. VeChainThor operates with a fixed number of sealers (size up to N) in a specific period of time, denoted as $\{s_1, s_2, \dots, s_N\}$. Sealers may join or leave the system, so N may change. Among them, at most f are Byzantine and fail arbitrarily. VeChainThor assumes $N \geq 3f + 1$. In our attack, we assume that f sealers are Byzantine.

Security goals. VeChainThor claims to achieve *consistency* and *liveness* [51], as defined below.

- **Consistency.** If an honest sealer finalizes block B , another honest sealer finalizes B' such that $h(B) = h(B')$, $B = B'$.
- **Liveness.** The finalized chain eventually grows.

Note that VeChainThor’s security goals differ from those in classical BFT protocols. The conventional definition of liveness requires that every transaction issued by an honest party will eventually be finalized. In contrast, VeChainThor adopts a weaker notion, merely requiring that the finalized chain grows. Our attacks prevent blocks from being finalized and thus violate both definitions.

3 Review of VeChainThor Protocol

VeChain has evolved through two major versions: *VeChain1.0* [40] and *VeChain2.0* [42]. VeChain 1.0 implements the Proof-of-Authority PoA Aura protocol [31]. In particular, sealers take turns to propose blocks. No *voting* process is involved. Instead, a proposal that extends a chain can be viewed as a vote to the blocks on the chain. For a block B , if a chain that extends B have at least $N/2$ blocks proposed by distinct sealers, then B is finalized.

VeChainThor [51], the consensus protocol of VeChain2.0, improves Aura via two optimizations. VeChainThor introduces a finality mechanism called *finality with one bit* (FOB). FOB ensures that once a block is finalized, its order will never be reversed. Additionally, VeChainThor uses a *verifiable random function* (VRF)-based randomness beacon to select the proposers of the blocks.

3.1 Notations and Definitions

Epoch and slot. Time is divided into epochs. Each epoch has $E = 180$ slots. Each slot lasts for a duration of $\tau = 10$ seconds. The slot number begins with 0 and increments monotonically. Given slot number t , one can obtain the epoch number via $\Omega(t) \leftarrow \lfloor \frac{t}{E} \rfloor$.

Block; block tree; conflicting; canonical chain. Blocks are the basic data structures. Each block is in the form of $\langle \text{slot}, \text{parent}, \text{time}, \text{batch}, \text{com}, \text{awn} \rangle$, where *slot* is the slot number, *parent* is the hash of the parent block, *time* is a timestamp, *batch* consists of a batch of transactions, *com* denotes a single-bit vote, and *awn* is an *accumulated witness number*. Later, we will define how *awn* is calculated. The *com* field is unique in VeChainThor, which has one bit. If a

block B proposed by a sealer s_i has its *com* field as "1" for a checkpoint block B' , we say that s_i votes for B' . By default, the first block proposed in each epoch is called a *checkpoint block*. We use CP to denote the latest checkpoint block in the canonical chain.

Every sealer uses a block tree $\mathcal{T}$ to denote its received blocks. The root of the tree is the *genesis* block, i.e., the first block in the system. We use $C(B)$ to denote the branch/chain led by block B , and we call B the *head* of the chain. Given $C(B)$, the height of B is the number of blocks from the genesis block to B , denoted as $h(B)$. Block B extends B' if B' is on the branch led by B . Blocks B and B' are *conflicting* if neither B extends B' nor B' extends B . Two chains $C(B_1)$ and $C(B_2)$ are *conflicting*, if there exist $B'_1 \in C(B_1)$ and $B'_2 \in C(B_2)$ such that B_1 and B_2 are conflicting. Among the branches in the block tree, $\mathcal{T}$, one of them is the *canonical chain*.

Randomness beacon. VeChainThor uses the randomness beacon [12] to generate a stream of randomness. Given an epoch e , the randomness beacon b_e is calculated using Q consecutive blocks from epoch $e - 2$. In particular, the beacon value is calculated as $b_e = \text{Hash}(\beta(B_{e-2,1}) \parallel \beta(B_{e-2,2}) \parallel \dots \parallel \beta(B_{e-2,Q}))$, where $\{B_{e-2,1}, \dots, B_{e-2,Q}\}$ are Q consecutive blocks in epoch $e - 2$ and $\beta(B)$ is the random seed of block B . The random seed of block B is generated by the proposer via VRF [25] (cf. Appendix A) using the seed of B' as input, where B' is the parent block of B . Under such settings, the value of the randomness beacon can be predicted two epochs ahead of time. Our attack does not manipulate the randomness beacon [26] and assumes that the randomness beacon is bias-resistant [10, 12, 24].

Proposer, verifier, and active sealers. In VeChainThor, each sealer has two roles: *proposer* and *verifier*. Proposers propose blocks while verifiers validate them. One proposer is eligible to propose a block in each slot. The proposer is selected using:

$l_t = \min_{u \in \text{Active}_i} \text{Hash}(b_e \parallel t \parallel \text{Active}_i)$, where b_e is the randomness beacon for epoch e , t is the slot number, and Active_i is the set of *active* sealers. In particular, each sealer s_i maintains a set Active_s of active sealers. By default, Active_i consists of all N sealers. The list is updated in every slot. In some slot t , if a sealer s_i does not receive a block from the eligible proposer s_j , s_i removes s_j from Active_i . Later, if s_i receives a block from s_j , it will add s_j back to Active_i . After calculating l_t , the active sealer with the smallest hash value is selected as the proposer.

Justification and finalization. Similar to the friendly finality gadget [6], VeChainThor considers the "justify-finalize" relationship for checkpoint blocks. In particular, a checkpoint B is *justified* if more than $N - f$ distinct sealers have voted for B . B is *finalized* if another checkpoint block that extends B is justified. By default, the genesis block is already finalized.

Definition 1 (Justified checkpoint, JC). A checkpoint B with height $h(B)$ is a *justified checkpoint* (JC) if $N - f$ distinct sealers have voted for B by setting the *com* field of their block proposers as "1".

Definition 2 (Finalized checkpoint, FC). A block B at height $h(B)$ becomes *finalized* (FC), if $N - f$ distinct sealers have voted for block B' such that: (1) B' extends B ; and (2) B was previously justified.

Accumulated witness number (AWN) [19]. The AWN for a block denotes the number of active sealers that might have received the

block. Specifically, AWN (denoted as π) for a block B is calculated as $\pi_B = \pi_{B'} + |\text{Active}_i|$, where $\pi_{B'}$ is the AWN of B 's parent block B' and $|\text{Active}_i|$ represents the number of active sealers based on the Active_i of s_i . AWN is updated in every slot. When a sealer proposes a block, it adds the AWN in its block. Upon receiving a block, each sealer also verifies whether the value included in the block matches its local AWN.

Algorithm 1 Chain selection rule for sealer s_i

Input: The block tree $\mathcal{T}$

Output: The canonical chain $C(B)$

```

1: Select the branches  $C = \{C(B_1), \dots\}$  with the largest JCH and prune
   other branches from  $\mathcal{T}$ 
2: if  $|C| > 1$  then
3:   Select the branches with the highest AWN and filter other chains
   from  $C$ 
4:   if  $|C| > 1$  then
5:     Select the longest chain in  $C$  and filter other branches
6:   end if
7: end if
8: return  $C(B)$  where  $C = \{C(B)\}$ 

```

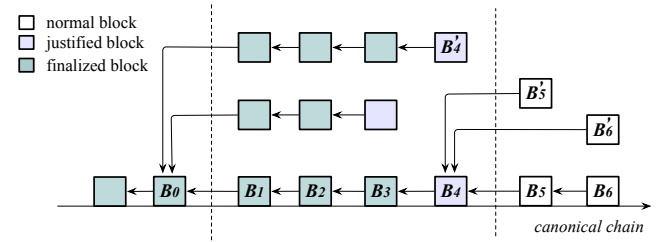


Figure 1: Example of the chain selection rule. Among all the branches, the branches led by block B'_4 , B'_5 , B_6 , and B'_6 all have the same justified checkpoint height (JCH), as the blocks B_4 and B'_4 have the same height. Among the four branches with the same JCH, the branch led by B_6 has the largest accumulated witness number (AWN). $C(B_6)$ is then the canonical chain and B_6 is the head.

Chain selection rule. The chain selection rule selects one branch from the block tree $\mathcal{T}$ as the canonical chain. We show an example in Figure 1. First, the rule selects the branch with the largest *justified checkpoint height* (JCH), where JCH denotes the height of the last justified checkpoint block. Second, if multiple branches have the same JCH, select the chain with the largest AWN. Third, if multiple branches share the same AWN, select the longest chain.

Voting rule. The voting rule is unique in VeChainThor. Informally, the proposer s_i may propose a block that does not extend its canonical chain. The proposer then uses the *com* field to represent whether it extends its canonical chain. A vote *com* = 1 means s_i extends its canonical chain and *com* = 0 otherwise.

We show the voting rule in Algorithm 2. The voting rule takes input chain $C(B_0)$ (the canonical chain of a sealer s_i) and outputs $(v(C(B_0)), B)$. Here B might be $\perp$ or a block conflicting with B_0 . If $B = \perp$, s_i has not received a longer but conflicting chain with its canonical chain. If s_i has another branch $C(B)$ in its block tree such that $C(B)$ is conflicting with its canonical chain, its vote is set as 0.

Algorithm 2 Voting rule for sealer s_i

Input: Chain $C(B_0)$
Output: A single-bit vote $v(C(B_0))$

```

1: Let  $LJ$  be the last justified block in  $C(B_0)$ 
2: if there exists a block  $B \in \mathcal{T}$  such that JCH of  $C(B)$  is higher than  $h(LJ)$  and  $C(B)$  is conflicting with  $C(B_0)$  then  $\triangleright h(LJ)$  means the height of the last justified block
3:    $v(C(B_0)) \leftarrow 0$ 
4: else
5:    $v(C(B_0)) \leftarrow 1$ 
6: end if
7: return  $(v(C(B_0)), B)$ 
```

3.2 Workflow of VeChainThor

We show the pseudocode of the VeChainThor protocol in Algorithm 3. The protocol proceeds as follows.

Algorithm 3 The VeChainThor protocol for slot t

```

1: Each sealer  $s_i$  determines its role in the slot  $t$  based on the randomness beacon. Let  $T_0$  be the time slot  $t$  begins. Let  $LJ$  be the last justified block. Let  $\mathcal{T}$  be the block tree.
   $\triangleright$  Block Proposal:
2: if  $s_i$  is the proposer then
3:   At time  $T_0$ , query Algorithm 1 and obtain the canonical chain  $C(B_0)$ , where  $B_0$  is the head of the canonical chain; query Algorithm 2 using  $C(B_0)$  as input and obtain  $(v(C(B_0)), B')$ 
4:   if  $B' \neq \perp$  then
5:      $parent \leftarrow \text{Hash}(B')$ 
6:   else
7:      $parent \leftarrow \text{Hash}(B_0)$ 
8:   end if
9:   Create a block  $B = \langle t, parent, T_0, txs, v(C(B_0)), awn \rangle$  and then broadcast  $B$ , where  $txs$  consists of a batch of transactions from mempool and  $awn$  is its AWN
10: end if
   $\triangleright$  Block Verification:
11: upon receiving a valid block  $B$  from the proposer  $s_j$  do
12:   After checking the received block, update the block tree such as  $\mathcal{T} \leftarrow \mathcal{T} \cup B$ 
   $\triangleright$  Chain Finalization:
13: if  $t$  is the first slot of the current epoch then
14:   Let  $B^*$  be the checkpoint block of the last epoch
15:   Calculate the  $S$  as the sum of the votes for  $B^*$  (the sum of the com field of the proposed blocks of epoch  $e - 1$  where the proposers are distinct)
16:   if  $S \geq N - f$  then
17:     Mark  $B^*$  as justified and set  $LJ \leftarrow B^*$ 
18:     if there exists a justified block  $B'$  on the branch led by  $B^*$  then
19:       Finalize  $B'$  and all the blocks led by  $B'$ 
20:     end if
21:   end if
22: end if
```

Block proposal. In each slot t , each sealer determines its role based on the randomness beacon. We show the workflow in Figure 2. Let T_0 be the time slot t starts. At T_0 , if s_i is the proposer, it starts to generate block. Specifically, s_i first queries Algorithm 1 to obtain

the canonical chain $C(B_0)$ and then queries Algorithm 2 to obtain a vote $v(C(B_0))$, B' (line 3, algorithm 3). As mentioned in §3.1, B' might be a block that is conflicting with s_i 's canonical chain. If such a conflicting chain exists, s_i sets B' as the parent block. Otherwise, s_i sets the head of its canonical chain as the parent block. Afterward, s_i creates a new block B and sends B to all sealers (line 4, algorithm 3).

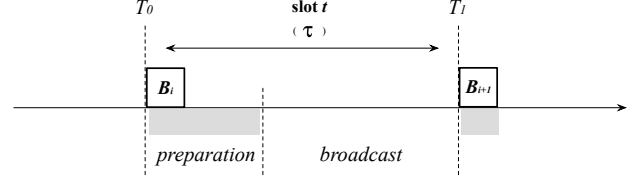


Figure 2: Each slot consists of two phases. At the beginning of slot t (T_0), the proposer creates a block. Before the end of slot t (T_1), the proposer sends the block to all active sealers.

Block verification. Upon receiving a block B from the proposer s_j of slot t , s_i verifies whether B is valid as follows: (i) s_j is a valid proposer; (ii) B consists of a valid signature from s_j ; and (iii) whether the timestamp of B is between $[T_0, T_0 + \tau]$. If B is valid, s_i adds B to its block tree $\mathcal{T}$ (line 9, algorithm 3).

Chain finalization. In the first slot of every epoch, each sealer s_i updates its justified blocks and finalized blocks in $\mathcal{T}$. Given the last checkpoint block B^* from the last epoch, s_i checks whether $N - f$ sealers have voted for B^* . If so, B^* is justified (line 12, algorithm 3). If B^* extends a justified block B' , all blocks led by B' are finalized (line 13-14, algorithm 3). To count the number of votes, s_i calculates the sum of “1” in the *com* field of the proposed blocks that extend B^* , where these blocks are proposed by distinct sealers. By adopting FOB, if a block B is extended by $4f + 1$ blocks, B is expected to be finalized.

4 The Greedy Attack**4.1 Overview of Our Attack**

In this paper, we present an attack called *greedy attack*, a reorganization attack to the VeChainThor protocol. We focus on the FOB mechanism and study whether FOB can finalize blocks as VeChainThor claims. We show that under our attack, the finalization guarantee and the expected latency of VeChainThor are even worse than those of PoA Aura.

What can be achieved for the PoA Aura protocol? In Aura, it is mentioned that if a block B has been extended by $N/2$ blocks, B is finalized. Aura is susceptible to reorganization attacks. A known example is the time-manipulation attack [49], where one block is reorganized each time. Even if f blocks are reorganized, for any block B that is not reorganized, Aura can still finalize B after N slots. To see why this is the case, consider $N = 3f + 1$ and the following case: $\{s_1, \dots, s_{f+1}\}$ are honest, $\{s_{f+2}, \dots, s_{2f+1}\}$ are Byzantine, and $\{s_{2f+2}, \dots, s_{3f+1}\}$ are honest. We consider the extreme case where blocks proposed by $\{s_{2f+1}, \dots, s_{3f+1}\}$ are reorganized. To reorganize these blocks, blocks proposed by $\{s_{f+2}, \dots, s_{2f+1}\}$ have to be released. We now consider the block B proposed by s_1 in slot t . By the end of slot $t + 3f$, B has been extended by at least

$N - f$ blocks, exceeding the $N/2$ threshold to finalize B . Currently, VeChainThor has $N = 101$ sealers [37], so the time to finalize block B is 68 slots. For other blocks that are not reorganized, the expected time to finalize each block is the same. We provide more analysis in Appendix C).

What can be achieved for the VeChainThor protocol, after the optimizations on top of Aura? We show that after the optimizations VeChainThor applies to Aura, reorganization attacks can still be found in VeChainThor. We claim the following result for our greedy attack.

Under greedy attack, about 28.57% blocks are expected to be reorganized. A block that is not reorganized is expected to be finalized with only 0.19% probability, i.e., the expected duration to finalize the block is 526 slots.

Our greedy attack exploits the random proposer selection mechanism from VRF. Since proposers are randomly selected by VRF, the number of blocks that can be reorganized largely depends on the order of the proposers. We thus provide a greedy strategy where Byzantine sealers maximize the number of blocks that can be reorganized. We then perform a statistical analysis and show that 28.57% of the blocks are expected to be reorganized. We further extend the study and show that a block proposed by an honest sealer can be finalized with a probability of only 0.19%. The expected latency of finalizing a block is thus an order of magnitude lower than the guarantee of Aura.

4.2 Warm-Up: From 1-reorg to k -reorg

We begin with a warm-up attack, where a single Byzantine sealer s_1 can launch the attack to reorganize one block at a time. We call it 1-reorg attack. Let slot t be the slot in which s_1 is an eligible proposer. The attack strategies are summarized as follows.

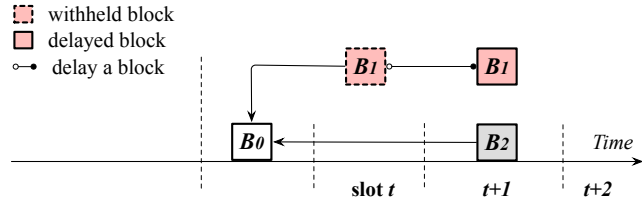


Figure 3: Illustration of the 1-reorg attack. In slot t , a Byzantine sealer s_1 creates a valid block B_1 and withholds B_1 . In slot $t+1$, the proposer is honest and it proposes block B_2 that also extends B_0 . Before slot $t+2$ begins, s_1 releases B_1 . According to the chain selection rule, B_1 has a higher accumulated witness number (AWN). Accordingly, all honest sealers will set the chain led by B_1 as the canonical chain.

- At the beginning of slot t , s_1 executes the chain selection rule to obtain a head block B_0 . It creates a block B_1 by setting B_0 as the parent block and then withholds B_1 .
- Before slot $t+2$ begins, s_1 releases B_1 .

We present an example in Figure 3, where block B_2 proposed by an honest sealer is reorganized. Here, we explain why the Awn of B_1 is larger and why B_2 is reorganized.

- Let N be the number of active sealers when B_0 is proposed, i.e., all N sealers actively participate in the protocol and propose their blocks.

- When B_1 is proposed by s_1 , it sets its *awn* as $\pi_{B_0} + N$ in its block B_1 and withholds B_1 .
- Since s_2 (the proposer of slot $t+1$) does not receive B_1 , it removes s_1 from *Active_i* and sets the *awn* field as $\pi_{B_0} + N - 1$ in its block.
- After B_1 is released, other sealers can verify the Awn (as they have all sealers in their lists of active sealers). However, the Awn in block B_2 is lower. Accordingly, block B_2 is reorganized, and B_1 is the canonical chain.

We can also generalize the 1-reorg attack to k -reorg attack by increasing the number of slots Byzantine sealers controls. That is, if the proposers in the k consecutive slots are Byzantine and the sealers of the subsequent k slots are honest, the Byzantine sealers can withhold their blocks. Let slot t be the first slot where the attack begins. The Byzantine sealers can release their blocks before slot $t+2k$ begins. We show in Figure 4 an example of 3-reorg attack. Following a similar argument of 1-reorg attack, the k blocks proposed by honest sealers will be reorganized.

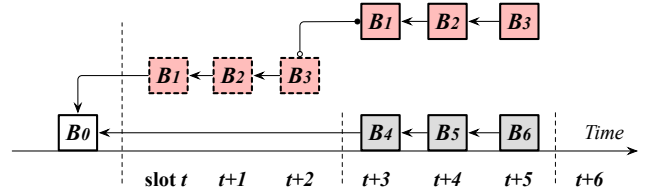


Figure 4: Illustration of the k -reorg attack where $k = 3$. The proposers of slot t to $t+2$ are Byzantine and the proposers of slot $t+3$ to $t+5$ are honest. Byzantine proposers withhold their blocks (B_1 to B_3) and release them before slot $t+6$ begins. According to the chain selection rule, B_1 to B_3 become the canonical chain and B_4 to B_6 are discarded.

This k -reorg attack is conceptually similar to the selfish mining attack [17], an attack originally known to PoW and later found for Proof-of-Stake (PoS) protocols [11, 27, 30]. We do not claim too much novelty for our warm-up attack. Instead, we use it as the baseline to better explain our greedy attack.

4.3 The Greedy Attack

We now extend our warm-up attack to a greedy version where Byzantine sealers choose their attack strategies *greedily*. Namely, VeChainThor uses VRF to select proposers. Byzantine sealers cannot specify a fixed k to launch the attack. As a solution, Byzantine sealers can *predict* their own positions in each epoch and then greedily choose the corresponding k to maximize the number of reorganized blocks. Currently, the randomness beacon of VeChainThor can be known two epochs in advance (see §3.2), so Byzantine sealers can predict their positions. In fact, even if the randomness beacon cannot be known in advance to all sealers, Byzantine sealers can still calculate their VRF values in advance to predict their positions. Our attack consists of two phases as follows.

4.3.1 Planning Phase. Given any epoch, let t be the first slot of the epoch. Initiate an initiator array $I \leftarrow [\perp]^E$ and then iterate slot number $t' \in [t, t+E]$ as follows.

- If the eligible proposer in slot t' is Byzantine, calculate k_1 such that the proposers in slot $[t', t' + k_1 - 1]$ are Byzantine. Then

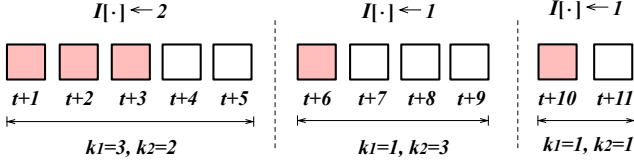


Figure 5: Illustration of the planning phase. For each slot t' , the algorithm determines k_1 as the number of consecutive Byzantine proposers and k_2 as the number of subsequent consecutive honest proposers. The initiator array I is then updated according to $\min(k_1, k_2)$.

calculate k_2 such that the proposers in $[t' + k_1, t' + k_1 + k_2 - 1]$ are honest.

- If $k_2 - k_1 \geq 0$, set $I[t'] \leftarrow k_1$. Otherwise, set $I[t' + k_1 - k_2] \leftarrow k_2$.
- Set t as $t' + k_1 + k_2$ and continue the iteration.

At the end of the iteration, the initiator array is filled with values.

4.3.2 Execution Phase. When the actual epoch begins, the Byzantine sealers perform the following strategy.

- In the block proposed by any Byzantine proposer, set $v(B^*)$ as 0, where B^* is the latest checkpoint block.
- At the beginning of any slot t , check $I[t]$. If $I[t] \neq \perp$, let k be $I[t]$ and then perform a k -reorg attack.

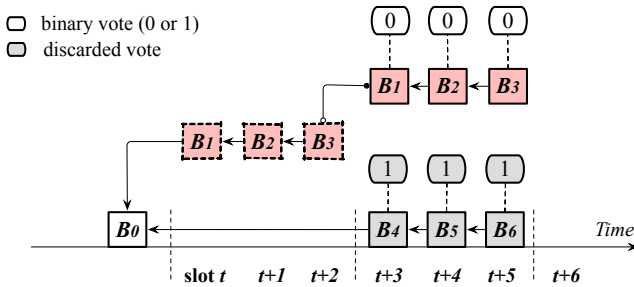


Figure 6: Illustration of the greedy attack. The Byzantine sealers set their votes as "0" in B_1 to B_3 . Honest sealers set their votes as "1" in blocks B_4 to B_5 . After B_1 to B_3 are released, they become the canonical chain. The votes from honest sealers are not counted.

Briefly speaking, given a slot t' , the attack strategy first predicts the number of k_1 consecutive slots such that the proposers are Byzantine and the subsequent k_2 consecutive slots such that the proposers are honest. If $k_2 - k_1 \geq 0$, the scenario matches the k_2 -reorg attack mentioned above. In this case, a k_1 -reorg attack can be launched at the beginning of slot t' . If $k_2 - k_1 < 0$, the maximum number of reorganized blocks is bounded by k_2 . In this case, the attack can be launched later (i.e., slot $t' + k_1 - k_2$), and the adversary can launch a k_2 -reorg attack.

It now becomes clear why we call our attack greedy attack. Namely, our attack is similar to the decision-making process in greedy algorithms, where optimal decisions are made step-by-step to maximize immediate results [22].

4.4 Why does Greedy Attack Work?

Our greedy attack consists of an adaptive reorganization strategy and a "no-vote" strategy. One may wonder why we design such an attack strategy and why this attack can significantly lower the finality guarantees. Briefly speaking, our greedy attack tries to maximize the number of reorganized blocks by honest sealers and also minimize the number of votes to the latest checkpoint block B^* . Namely, the votes included in the reorganized blocks will not be counted. If Byzantine sealers do not vote, the number of votes from distinct sealers for B^* will not exceed $N - f$, making it impossible to justify B^* .

Formally, we define an RB parameter as the total number of reorganized blocks in an epoch. In particular, given the indicator I in an epoch, RB is the sum of $I[t]$ for any $I[t] \neq \perp$. In an epoch, there are E slots, and we need at least $N - f$ matching votes from distinct sealers to justify a block. Since RB blocks are reorganized and f Byzantine sealers do not vote, the maximum number of matching votes that can be collected is bounded by $E - RB - F$, where F is the number of Byzantine proposers in the epoch.

As VeChainThor assumes $N \geq 3f + 1$, i.e., $f/N \approx 1/3$, we can get $\mathbb{E}(F) = \frac{(E-RB)}{3}$ and thus the number of slots in which the proposers are honest is expected to be $\frac{2(E-RB)}{3}$. We further need to ensure that among the $\frac{2(E-RB)}{3}$ slots, there are at least $N - f$ distinct proposers. We are then left to analyze the expected value of RB and the probability that there are $N - f$ distinct honest proposers among $\frac{2(E-RB)}{3}$ slots. Given there are $E = 180$ slots in one epoch, we show that such probability is negligible ($\leq 0.19\%$).

Economic implications. Beyond undermining finality guarantees, the greedy attack also compromises the protocol's incentive compatibility. Honest proposers lose both block rewards and transaction fees, whereas the adversary incurs no penalty, since no slashing conditions are triggered.

5 Attack Analysis

In this section, we provide a statistical analysis of our attack. In particular, we analyze the probability p_f of finalizing a checkpoint block B^* (proposed by an honest sealer). We call p_f the *finalization probability*. As mentioned in §4, at least $N - f$ distinct proposers have to be honest in $\frac{2(E-RB)}{3}$ slots to justify a block. Additionally, another block that extends B^* needs to be justified before B^* can be finalized. Our analysis thus consists of three steps. First, we analyze the expected value of RB . Second, we analyze the probability of justifying a block p_j . Finally, we can calculate the finalization probability p_f .

5.1 Analysis using Current VeChainThor Setup

VeChainThor currently has $E = 180$ and $N = 101$ (i.e., $f = 33$ and $N = 3f + 2$). Our theoretical analysis focuses on this setting and assumes f sealers are Byzantine, i.e., the Byzantine sealers control 33.33% of the sealers.

Theorem 1. *Given $E = 180$, the expected value of RB (the total number of reorganized blocks in an epoch) is 51.*

Proof (Theorem 1). Following the notations used in §4, we analyze RB as follows. We fix k_2 and analyze whether the k_2 blocks

by honest proposers can be finalized. Let B_j be the first block proposed by an honest proposer so the blocks by honest proposers are $\{B_j, B_{j+1}, \dots, B_{j+k_2-1}\}$. There are two sub-cases.

Sub-case 1: Block B_{j+k_2} is proposed by an honest sealer. In this case, these k_2 blocks can be finalized only if $k_1 = k_2$, i.e., there are exactly k_2 Byzantine proposers before these honest proposers.

Sub-case 2: Block B_{j+k_2} is proposed by a Byzantine sealer. In this case, these k_2 blocks can be finalized only if $k_1 \geq k_2$, i.e., there are at least k_2 proposers prior to these honest proposers.

We then define the following events.

- X_{k_2} represents the event where the k_2 blocks are reorganized;
- $X_{k_2}^1$ denotes sub-case 1;
- $X_{k_2}^2$ denotes sub-case 2.

We thus have:

$$\Pr(X_{k_2}) = \Pr(X_{k_2}^1 \cup X_{k_2}^2) = \Pr(X_{k_2}^1) + \Pr(X_{k_2}^2) - \Pr(X_{k_2}^1 \cap X_{k_2}^2),$$

where the event $X_{k_2}^1 \cap X_{k_2}^2$ represents

$\{\text{blocks } B_j, B_{j+1}, \dots, B_{j+k_2-1} \text{ and } B_{j-k_2-1} \text{ are honest}\} \wedge \{\text{blocks } B_{j-k_2}, B_{j-k_2+1}, \dots, B_{j-1} \text{ and } B_{j+k_2} \text{ are Byzantine}\}$. We can then calculate $\Pr(X_{k_2})$ as follows.

$$\begin{aligned} & \Pr(X_{k_2}^1) + \Pr(X_{k_2}^2) - \Pr(X_{k_2}^1 \cap X_{k_2}^2) \\ &= \frac{2}{3} \cdot \left(\frac{1}{3} \cdot \frac{2}{3}\right)^{k_2} + \frac{1}{3} \cdot \left(\frac{1}{3} \cdot \frac{2}{3}\right)^{k_2} - \frac{2}{3} \cdot \frac{1}{3} \cdot \left(\frac{1}{3} \cdot \frac{2}{3}\right)^{k_2} \\ &= \left(\frac{2}{9}\right)^{k_2} - \left(\frac{2}{9}\right)^{k_2+1} \end{aligned}$$

Since $E = 180$, the expected value of RB is:

$$\begin{aligned} 180 \cdot \mathbb{E}(X_{k_2}) &= 180 \cdot \sum_{k_2=1}^{\infty} k_2 \cdot \Pr(X_{k_2}) \\ &= 180 \cdot \sum_{k_2=1}^{\infty} k_2 \cdot \left(\left(\frac{2}{9}\right)^{k_2} - \left(\frac{2}{9}\right)^{k_2+1} \right) \quad \square \\ &= 180 \cdot \sum_{k_2=1}^{\infty} \left(\frac{2}{9}\right)^{k_2} = 180 \cdot \frac{2}{7} \\ &= 51 \end{aligned}$$

Corollary 1. Given $E = 180$, 28.57% of the blocks will be reorganized under our greedy attack.

Proof. According to the proof of Theorem 1, $\frac{2}{7} \approx 28.57\%$ of the blocks are expected to be reorganized.

Lemma 1. Given $N = 101$ and an unbiased randomness beacon, the probability that a sealer is selected as a proposer at least once among k slots is $p = 1 - \left(\frac{100}{101}\right)^k$.

Theorem 2. Given $E = 180$ and $N = 101$, the probability that a checkpoint block can be justified is 0.0438.

Proof (Theorem 2). As mentioned in §4, we need to ensure that the number of distinct proposers in $\frac{2(E-RB)}{3}$ slots is greater than $N - f$ to finalize a checkpoint block. According to Theorem 1, in each epoch, $RB = 51$. As $E = 180$ and $N = 101$, we need to analyze the probability that among $\frac{2(E-RB)}{3} = 86$ slots, there are at least 67 distinct honest eligible proposers.

From Lemma 1, the probability p that any particular sealer is selected at least once in these $k = 86$ slots is $1 - \left(\frac{100}{101}\right)^{86}$. We can then model the number of sealers that are selected at least once as a binomial distribution $\mathcal{B}(101, p)$. The probability that there are at least 67 distinct honest proposers can be modeled as $p_j = \sum_{x=N-f}^N \binom{101}{x} p^x (1-p)^{101-x} = \sum_{x=67}^{101} \binom{101}{x} p^x (1-p)^{101-x}$. By checking a standard binomial distribution table, we have $p_j = 0.0438$.

Theorem 3. Given $E = 180$ and $N = 101$, the probability that a checkpoint block is finalized (finalization probability) p_f is 0.0019.

Proof (Theorem 3). For a block B^* to be finalized, it has to be justified first, and another block that extends B^* needs to be justified. Accordingly, the finalization probability $p_f = p_j^2 = 0.0019$. $\square$

6 Implementation and Evaluation

Implementation. We implement our attacks based on the source code of VeChainThor¹. To implement our attack, we develop an *attacker simulator* and a *coordinator*. The attacker simulator is a custom-modified version of the VeChainThor codebase that executes our attack strategies. The *coordinator* manages the simulators of all Byzantine sealers so Byzantine sealers are aware of their positions in each epoch (i.e., the slots in which the proposers are Byzantine), and they all take the same actions during the attack.

Our codebase modifies about 4,000 lines of code on top of the source code. We have made our codebase and experimental results publicly available².

Experimental setup. We deploy a local network to evaluate our attack. Our experiments use the default configurations of VeChainThor. We deploy 101 sealers [39]. Among them, 33 are Byzantine sealers, and 68 are honest. In particular, sealers with the identifiers $\{33, 34, \dots, 65\}$ are Byzantine. Such a setup matches the configuration of the production system [39]. VeChainThor uses a peer-to-peer network for the underlying network communication. We set up a full connection mode in our experiments, where every pair of sealers is directly connected. We use Amazon EC2 for our evaluation, and we launch all sealers in one c6a.16xlarge instance. The c6a.16xlarge instance has 64 virtual CPUs and 128GB of memory.

Our experiment lasts for 20 epochs, i.e., about 10 hours. We launch our attack in epoch 3. We report the number of blocks proposed by Byzantine sealers and honest sealers to assess the number of finalized blocks. As a comparison, we set a benchmark and evaluate the results when no attack is launched. We consider it a baseline.

Evaluation results. We now report the evaluation results of our attacks. We assess both scenarios for our benchmark (no attack is launched) and under attack. As shown in Figure 7, when no attack is launched, blocks continue to be finalized, and the height of the finalized blocks grows by 180 in each epoch.

In contrast, under our greedy attack, the number of finalized blocks drops significantly. We show the result in Figure 7. In this case, starting from epoch 4, the height of the last finalized block does not increase (e.g., the height of the finalized block in both

¹VeChainThor codebase: <https://github.com/vechain/thor/tree/138d4f66d9>

²Our codebase: <https://zenodo.org/records/18344489>.

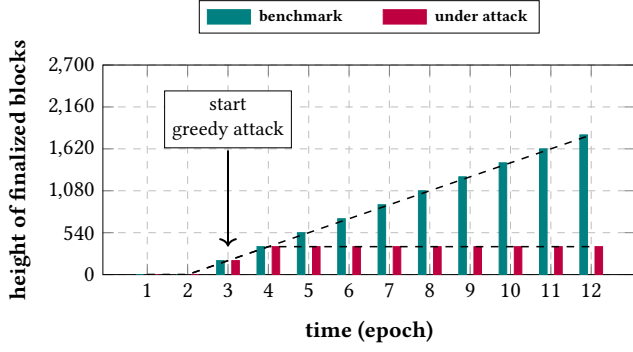


Figure 7: The height of the finalized blocks in the failure-free case (benchmark) and under greedy attack.

epoch 4 and epoch 5 is 360). This is expected. Namely, we corrupt f sealers 33, 34, ..., 65, and f blocks by honest sealers are reorganized under our attack. As a result, at most $f + 1$ distinct sealers can vote under greedy attack. Hence, no blocks can be finalized. This result matches our theoretical analysis.

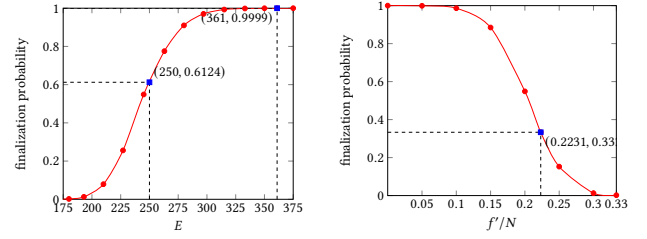
7 Security Boundary and Mitigation

Although VeChainThor improves the round-robin approach (by Aura) and uses VRF to select proposers, reorganization attacks still exist. Byzantine sealers can still collude to learn their positions in the slots to launch the attack. In fact, under the current configuration, the VRF mechanism makes it more challenging to collect $N - f$ matching votes from distinct sealers during our attack (see Theorem 2). We define an attack as successful if blocks fail to be finalized. In the following, we analyze the closely related security parameters of VeChainThor and discuss mitigation solutions.

The security boundary of the epoch length E . Our analysis shows that the expected number of reorganized blocks is closely related to the epoch length E . We vary E and evaluate its effect on the finalization probability. As shown in Figure 8(a), the finalization rate improves as E increases. For example, when $E = 250$, the finalization probability is 0.6124, and when E increases to 280, the probability rises to 0.91008. This is expected. Namely, the fraction of blocks that can be reorganized in our attack is fixed. With more slots in each epoch, the probability that $N - f$ distinct proposers are honest increases. Under the current setting of $f/N \approx 1/3$, to ensure that the finalization probability is close to 1 (i.e., 0.9999), E must be set to at least 361.

The security boundary of the fraction of Byzantine sealers f' . We next fix $E = 180$ and vary the fraction of sealers controlled by the Byzantine sealers. Let f' denote the number of Byzantine sealers. We show f'/N vs. the finalization probability in Figure 8(b). As shown in the figure, the finalization probability increases as f'/N decreases. For example, the finalization probability is 0.3333 when the Byzantine sealers control 22.31% of the sealers. In contrast, the probability increases to 0.88478 when the adversarial fraction is reduced to 16%. This result is consistent with prior observations of reorganization attacks: as f'/N grows, the number of blocks that can be reorganized increases, making our attack more effective.

Under the current setting of $E = 180$, to ensure that the finalization probability approaches 1 (e.g., 9992), f must not exceed 5%.



(a) The relationship between epoch length E and the probability of finalization under the current setting of $f/N \approx 1/3$. (b) The fraction of sealers controlled by the Byzantine sealers (i.e., f'/N) vs. finalization probability. E is fixed to 180.

Figure 8: Impact of epoch length and adversarial fraction on finalization probability.

To mitigate our attack, one straightforward approach is to enlarge E , i.e., the number of slots in each epoch. As shown in Figure 8(a), by setting E as 361, a block proposed by an honest sealer can be finalized with a probability of 0.9999. However, increasing E will significantly enlarge the finalization latency, which contradicts the design objective of the FOB mechanism, i.e., *fast* finality. A more sophisticated countermeasure is to adopt single secret leader election (SSLE) [4]. SSLE conceals the leader's identity until the leader publishes a block. This approach prevents Byzantine sealers from colluding in advance and thus mitigates our reorganization attacks. Beyond revising the randomness mechanism, one may need to revise the chain selection rule. Additionally, by having separate messages as votes, one may reuse the design principles of conventional Byzantine fault-tolerant (BFT) protocols [9, 46] to build a provably secure solution. As mentioned in Casper, the idea of the friendly finality gadget was inspired by BFT protocols.

8 Conclusion

We study VeChainThor, the consensus mechanism by VeChain. We present the *greedy attack*, a type of reorganization attack against VeChainThor, in which Byzantine sealers can greedily adjust their attack strategies to maximize the number of reorganized blocks. As a result, we show both theoretically and experimentally that the finality guarantee almost fails, and the expected time to finalize the block is significantly enlarged. In particular, every block proposed by an honest sealer can be finalized with only 0.19% probability.

Acknowledgment

This work was supported in part by National Natural Science Foundation of China under 62502266, Fundamental and Interdisciplinary Disciplines Breakthrough Plan of the Ministry of Education of China under JYB2025XDXM114, China Postdoctoral Science Foundation under 2023M741949, Theme-based Research Project T32-615/24-R from Hong Kong Research Grants Council, and Tsinghua Shuimu Scholar. Dr. Qin Wang was not financially supported. Dr. Haochen Wang completed this work while at Tsinghua University.

References

- [1] Sarah Azouvi, George Danezis, and Valeria Nikolaenko. 2020. Winkle: Foiling long-range attacks in proof-of-stake systems. In *ACM Conference on Advances in Financial Technologies (AFT)*. 189–201.
- [2] Christian Berger, Hans P Reiser, and Alysson Bessani. 2021. Making reads in BFT state machine replication fast, linearizable, and live. In *International Symposium on Reliable Distributed Systems (SRDS)*. IEEE, 1–12.
- [3] Binance Chain Developers. 2022. Binance Chain Whitepaper. <https://github.com/bnb-chain/whitepaper/blob/master/WHITEPAPER.md>. Accessed: Jan. 2025.
- [4] Dan Boneh, Saba Eskandarian, Lucjan Hanzlik, and Nicola Greco. 2020. Single secret leader election. In *Proceedings of the ACM Conference on Advances in Financial Technologies (AFT)*. 12–24.
- [5] Jonah Brown-Cohen, Arvind Narayanan, Alexandros Psomas, and S Matthew Weinberg. 2019. Formal barriers to longest-chain proof-of-stake protocols. In *ACM Conference on Economics and Computation (EC)*. 459–473.
- [6] Vitalik Buterin and Virgil Griffith. 2017. Casper the friendly finality gadget. *arXiv preprint arXiv:1710.09437* (2017).
- [7] Vitalik Buterin, Diego Hernandez, Thor Kamphofner, Khiem Pham, Zhi Qiao, Danny Ryan, Juhyeok Sin, Ying Wang, and Yan X Zhang. 2020. Combining GHOST and Casper. *arXiv preprint arXiv:2003.03052* (2020).
- [8] Christian Cachin, Rachid Guerraoui, and Luis Rodrigues. 2011. *Introduction to reliable and secure distributed programming*. Springer Science & Business Media.
- [9] Miguel Castro, Barbara Liskov, et al. 1999. Practical Byzantine fault tolerance. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, Vol. 99. 173–186.
- [10] Kevin Choi, Aathira Manoj, and Joseph Bonneau. 2023. SoK: Distributed Randomness Beacons. In *IEEE Symposium on Security and Privacy (S&P)*.
- [11] Francesco D'Amato, Joachim Neu, Ertem Nusret Tas, and David Tse. 2024. Goldfish: No More Attacks on Ethereum?! *International Conference on Financial Cryptography and Data Security (FC)* (2024).
- [12] Sourav Das, Vinith Krishnan, Irene Miriam Isaac, and Ling Ren. 2022. Spurt: Scalable distributed randomness beacon with transparent setup. In *IEEE Symposium on Security and Privacy (S&P)*. IEEE, 2502–2517.
- [13] Stefano De Angelis, Leonardo Aniello, Roberto Baldoni, Federico Lombardi, Andrea Margheri, Vladimiro Sassone, et al. 2018. PBFT vs proof-of-authority: Applying the CAP theorem to permissioned blockchain. In *CEUR Workshop Proceedings*, Vol. 2058. CEUR-Ws.
- [14] Sisi Duan, Hein Meling, Sean Peisert, and Haibin Zhang. 2014. BChain: Byzantine Replication with High Throughput and Embedded Reconfiguration. In *International Conference on Principles of Distributed Systems (OPODIS)*. 91–106.
- [15] Sisi Duan, Haibin Zhang, Xiao Sui, Baohan Huang, Changchun Mu, Gang Di, and Xiaoyun Wang. 2024. Dashing and Star: Byzantine Fault Tolerance from Weak Certificates. In *Eurosys*.
- [16] Cynthia Dwork, Nancy Lynch, and Larry Stockmeyer. 1988. Consensus in the presence of partial synchrony. *Journal of the ACM (JACM)* (1988).
- [17] Ittay Eyal and Emin Gün Sirer. 2018. Majority is not enough: Bitcoin mining is vulnerable. *Communications of the ACM (CACM)* 61, 7 (2018), 95–102.
- [18] Chen Feng and Jianyu Niu. 2019. Selfish mining in Ethereum. In *IEEE International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 1306–1316.
- [19] VeChain Foundation. 2025. Consensus Deep Dive. <https://docs.vechain.org/introduction-to-vechain/about-the-vechain-blockchain/consensus-deep-dive> Accessed: 2025-01-22.
- [20] Arthur Gervais, Ghassan O Karame, Karl Wüst, Vasileios Glykantzis, Hubert Ritzdorf, and Srdjan Capkun. 2016. On the security and performance of proof of work blockchains. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 3–16.
- [21] Rachie Guerraoui, Nikola Knežević, Vivien Quéma, and Marko Vukolić. 2015. The next 700 BFT protocols. *ACM Transactions on Computer Systems (TOCS)* 32, 4 (2015), 12:1–12:45.
- [22] Joseph B. Kruskal. 1956. On the shortest spanning subtree of a graph and the traveling salesman problem. <https://api.semanticscholar.org/CorpusID:120068278>
- [23] Rujia Li, Jingyuan Ding, Qin Wang, Keting Jia, Haibin Zhang, and Sisi Duan. 2025. Does finality gadget finalize your block? a case study of binance consensus. In *USENIX Security Symposium (USENIX Sec)*. 4109–4125.
- [24] Xuanji Meng, Xiao Sui, Zhaoxin Yang, Kang Rong, Wenbo Xu, Shenglong Chen, Ying Yan, and Sisi Duan. 2025. Rondo: Scalable and Reconfiguration-Friendly Randomness Beacon. In *The Network and Distributed System Security (NDSS) Symposium*.
- [25] Silvio Micali, Michael Rabin, and Salil Vadhan. 1999. Verifiable random functions. In *Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE, 120–130.
- [26] Ábel Nagy, János Tapolcai, István András Seres, and Bence Ladóczki. 2025. Forking the RANDAO: Manipulating Ethereum's Distributed Randomness Beacon. In *CCS*.
- [27] Joachim Neu, Ertem Nusret Tas, and David Tse. 2021. Ebb-and-flow protocols: A resolution of the availability-finality dilemma. In *IEEE Symposium on Security and Privacy (SP)*. IEEE, 446–465.
- [28] Joachim Neu, Ertem Nusret Tas, and David Tse. 2022. Two more attacks on proof-of-stake GHOST/Ethereum. In *ACM Workshop on Developments in Consensus (ConsensusDay)*. 43–52.
- [29] Michael Neuder, Daniel J Moroz, Rithvik Rao, and David C Parkes. 2020. Defending against malicious reorgs in Tezos Proof-of-Stake. In *ACM Conference on Advances in Financial Technologies (AFT)*. 46–58.
- [30] Michael Neuder, Daniel Moroz J, Rithvik Rao, and David Parkes C. 2021. Low-cost attacks on Ethereum 2.0 by sub-1/3 stakeholders. In *Proceedings of WINE Workshop on Game Theory in Blockchain*.
- [31] Ekparinya Parinya, Gramoli Vincent, and Jourjon Guillaume. 2020. The Attack of the Clones Against Proof-of-Authority. In *Annual Network and Distributed System Security (NDSS) Symposium*.
- [32] Rafael Pass and Elaine Shi. 2017. The sleepy model of consensus. In *International Conference on The Theory and Application of Cryptology and Information Security (ASIACRYPT)*. Springer, 380–409.
- [33] Ulysse Pavloff, Yackolley Amoussou-Guenou, and Sara Tucci-Piergiovanni. 2023. Ethereum proof-of-stake under scrutiny. In *ACM/SIGAPP Symposium on Applied Computing (SAC)*. 212–221.
- [34] Zhijie Ren and Ziheng Zhou. 2021. VIP214: Per-EPOCH VRF-based Random Beacon. <https://github.com/vechain/VIPs/blob/master/assets/vip214.pdf> (2021).
- [35] João Sousa, Eduardo Alchieri, and Alysson Bessani. 2014. State machine replication for the masses with BFT-SMArt. In *IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. 355–362.
- [36] Alistair Stewart. 2019. Poster: Grandpa finality gadget. In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2649–2651.
- [37] VeChain Documentation Team. 2025. Nodes. VeChain. <https://docs.vechain.org/core-concepts/nodes> Accessed on 2025-10-06.
- [38] Florian Tschorsch and Björn Scheuermann. 2016. Bitcoin and beyond: A technical survey on decentralized digital currencies. *IEEE Communications Surveys & Tutorials* 18, 3 (2016), 2084–2123.
- [39] Vechain. [n. d.]. Vechain: Web3 for Better. <https://vechain.org/>, Retrieved: Jul. 2024 [n. d.].
- [40] Vechain. 2018. Vechain Whitepaper 1.0 version. <https://files.vechain.org/vechaincontainer/whitepaper-1-0.pdf> (2018).
- [41] Vechain. 2018. Vechain Whitepaper 3.0 version. <https://vechain.org/wp-content/uploads/2023/10/vechain-whitepaper-3-0.pdf> (2018).
- [42] Vechain. 2024. Vechain Docs: Consensus Deep Dive. <https://docs.vechain.org/introduction-to-vechain/about-the-vechain-blockchain/consensus-deep-dive> (2024).
- [43] Qin Wang, Rujia Li, Qi Wang, Shiping Chen, and Yang Xiang. 2022. Exploring unfairness on proof of authority: Order manipulation attacks and remedies. In *ACM on Asia Conference on Computer and Communications Security (AsiaCCS)*. 123–137.
- [44] Gavin Wood. 2015. PoA Private Chains. <https://github.com/ethereum/guide/blob/master/poa.md>. Retrieved: Dec., 2021.
- [45] Gavin Wood et al. 2014. Ethereum: A secure decentralised generalised transaction ledger. *Ethereum project yellow paper* 151, 2014 (2014), 1–32.
- [46] Maofan Yin, Dahlia Malkhi, Michael K. Reiter, Guy Golan Gueta, and Ittai Abraham. 2019. HotStuff: BFT Consensus with Linearity and Responsiveness. *ACM Symposium on Principles of Distributed Computing (PODC)* (2019), 347–356.
- [47] Jianting Zhang, Wuhui Chen, Sifu Luo, Tiantian Gong, Zicong Hong, and Aniket Kate. 2024. Front-running Attack in Sharded Blockchains and Fair Cross-shard Consensus. In *The Network and Distributed System Security (NDSS) Symposium*.
- [48] Mingfei Zhang, Rujia Li, and Sisi Duan. 2024. Max Attestation Matters: Making Honest Parties Lose Their Incentives in Ethereum PoS. In *USENIX Security Symposium (USENIX Sec)*. 6255–6272.
- [49] Xinrui Zhang, Rujia Li, Qin Wang, Qi Wang, and Sisi Duan. 2023. Time-manipulation attack: Breaking fairness against proof of authority Aura. In *Proceedings of the ACM Web Conference (WWW)*. 2076–2086.
- [50] Xinrui Zhang, Qin Wang, Rujia Li, and Qi Wang. 2022. Frontrunning block attack in PoA clique: A case study. In *IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*. IEEE, 1–3.
- [51] Ren Zhijie and Zhou Ziheng. 2022. VIP-220: Finality with One Bit (FOB). <https://github.com/vechain/VIPs/blob/master/assets/vip220.pdf> (2022).

A Additional Details on the Randomness Beacon

A key component of VeChainThor consensus is the per-epoch randomness beacon, which determines the leader schedule and influences proposer selection across epochs. To ensure fairness and liveness, the beacon must remain unbiased, unpredictable, and publicly verifiable. VeChainThor implements this via a two-layer VRF-based design [34]: individual block proposers embed VRF-generated seeds

in their blocks, and these seeds are later aggregated to form the epoch-level beacon.

We summarize the procedure as follows (also in Figure 9):

- **VRF seed generation for a block.** For a proposed block B , the proposer takes the parent seed $\beta(B')$ and computes

$$(y, \pi) \leftarrow \text{VRF.Eval}_{sk}(\beta(B')),$$

where sk is the proposer's VRF secret key. The resulting y is set as the block's random seed $\beta(B)$, and the proof π (together with pk if needed) is included in the block. Validators verify correctness by checking

$$\text{VRF.Verify}_{pk}(\beta(B'), \beta(B), \pi) = \text{true}.$$

If verification fails, the block is invalid.

- **Epoch-level beacon aggregation.** To derive the randomness beacon b_e for epoch e , sealers collect Q consecutive blocks from epoch $e-2$,

$$\{B_{e-2,1}, B_{e-2,2}, \dots, B_{e-2,Q}\},$$

extract their seeds $\beta(B_{e-2,i})$, and compute

$$b_e = H(\beta(B_{e-2,1}) \parallel \beta(B_{e-2,2}) \parallel \dots \parallel \beta(B_{e-2,Q})).$$

Because b_e depends only on seeds from epoch $e-2$, its value becomes fixed once epoch $e-2$ finalizes. Thus, the beacon of epoch e can be predicted exactly two epochs in advance.

Workflow (Algorithm 4). In the *block proposal phase*, a proposer evaluates the VRF on the parent seed (line 1) and embeds the resulting output and proof into the block (line 2). In the *block validation phase*, validators check the correctness of the VRF proof (line 3) and reject invalid blocks (line 5). Finally, in the *epoch beacon phase*, the system aggregates Q seeds from epoch $e-2$ (line 7) and computes the beacon value b_e by hashing their concatenation (line 8), which is then returned as the randomness source (line 9).

Algorithm 4 VRF-based Randomness Beacon Generation

Input: Parent block B' , VRF keys (pk, sk) , epoch index e , parameter Q
Output: Block B with seed $\beta(B)$ and proof π , epoch beacon b_e

```

..... Block proposal .....
1:  $(y, \pi) \leftarrow \text{VRF.Eval}_{sk}(\beta(B'))$ 
2:  $\beta(B) \leftarrow y$ ; include  $(\beta(B), \pi, pk)$  into block  $B$  and send  $B$  to sealers
..... Block validation .....
3: Upon receiving  $B$ , check  $\text{VRF.Verify}_{pk}(\beta(B'), \beta(B), \pi)$ 
4: if verification fails then
5:   mark  $B$  as invalid
6: end if
..... Epoch beacon .....
7: Collect  $Q$  blocks from epoch  $e-2$ 
8: Compute  $b_e \leftarrow H(\beta(B_{e-2,1}) \parallel \dots \parallel \beta(B_{e-2,Q}))$ 
9: return  $(B, b_e)$ 

```

Properties. The VRF-based beacon achieves *bias resistance* (the beacon cannot be skewed by repeated trials, as each proposer's input is fixed to the parent seed), *unpredictability* (the beacon value b_e for epoch e remains unknown until epoch $e-2$ is finalized), and *public verifiability* (any node can validate the embedded VRF proof using the proposer's public key).

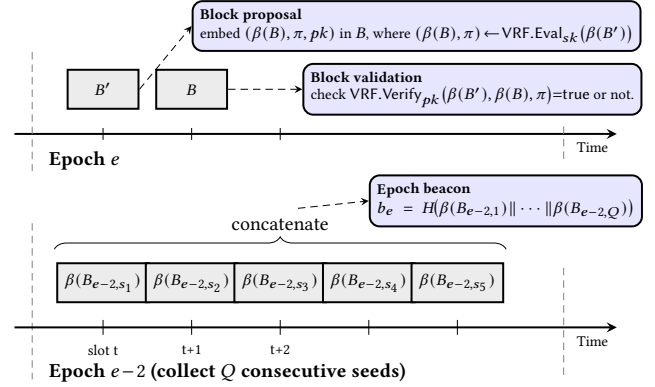


Figure 9: Beacon generation workflow: A proposer evaluates the VRF on the parent seed and embeds $(\beta(B), \pi)$, which validators verify; Q seeds from epoch $e-2$ are concatenated and hashed to obtain b_e .

B Related Work

Finality gadget. The notion of FFG was first proposed by Casper [6]. As mentioned in the introduction, FFG inherits the feature of Byzantine fault-tolerant (BFT) protocols [9, 15, 35, 46]. It directly supports one type of message called an *attestation*. In particular, every sealer can vote for a block via an attestation message. Most variants of FFG also adopt this pattern. Examples include Ethereum 2.0 [7], Polkadot [36], and BNB smart chain [3, 23]. Instead, VeChainThor embeds the votes in block proposals so it does not have attestation messages. Such a design also resembles a line of research on chain-based BFT protocols [14, 21].

Reorganization attacks. Reorganization (reorg) attacks have been extensively studied across different consensus paradigms. As mentioned previously, selfish mining attacks are arguably the first known reorganization attacks. Selfish mining was first identified in Bitcoin [17]. Selfish mining attack for Ethereum (ETH) 1.0 was later found [18], as Ethereum 1.0 also uses a variant of PoW. For Proof-of-Stake (PoS) protocol, selfish mining attacks are also found [5, 29]. Selfish mining attacks [17, 18] do not target security properties of the system. Instead, the goal is to make an adversary gain more rewards than its *fair* share, resembling the notion of reward fairness.

After Ethereum upgraded to 2.0, it now uses a PoS protocol as well. Several reorganization attacks have been found for Ethereum 2.0 [28, 33, 48]. For example, the bouncing attack [33] makes honest validators (i.e., sealers in the context of Ethereum) switch between two different branches so neither branch can be finalized. The liveness of the system is thus threatened. The staircase attack [48] targets the rewards of the validators. By launching the attack, the adversary continues to create forks. Hence, blocks and attestations by honest validators are discarded. If the adversary controls a large enough fraction of the stake, all honest validators will not receive any rewards. As mentioned before, our attack is also a reorganization attack. Instead of focusing on reorganizing blocks, our goal is to manipulate the voting bits and study the finality guarantee of VeChainThor.

Known attacks on PoA and variants. Vincent et al. [31] proposed the *cloning attack*. The idea is to clone the secret keys of a

Table 1: Comparison (selected) known attacks with ours.

Attack	Targeted blockchain		Attack requirement		Attack impact
	Consensus	Implementation	Adversary	NetworkConl	
Selfish mining [17]	PoW	Bitcoin	<33% mining	○	Honest miners' reward share decreases
ETH 1.0 selfish mining [18]	PoW	Ethereum 1.0	<33% mining	○	Honest miners' reward share decreases
Long-range attack [1]	PoS	Validator-based	$< \frac{N}{3}$	○	Finalized blocks can be reverted
Bouncing attack [33]	PoS	Ethereum 2.0	$< \frac{N}{3}$	○	Finality is delayed / weakened
GHOST avalanche attack[28]	PoS	Ethereum 2.0	<30% stake	○	Honest blocks suffer reorg losses
LMD balance attack[28]	PoS	Ethereum 2.0	<30% stake	●	Double spending is possible
Staircase attack [48]	PoS	Ethereum 2.0	<33% stake	○	Honest proposers' incentives are undermined
Time-manipulation [49]	PoA	Aura	$< \frac{N}{2}$	○	Honest proposers' reward share decreases
PoA Type-I [43]	PoA	Aura/Clique	$< \frac{N}{2}$	○	Honest transactions can be frontrun
PoA Type-II [43]	PoA	Aura/Clique	$< \frac{N}{2}$	○	Honest block proposers' reward share shrinks
Clique attack[50]	PoA	Clique	$< \frac{N}{2}$	○	Honest blocks may be rejected
Clone attack[31]	PoA	Aura/Clique	$< \frac{N}{2}$	●	Honest participants' view splits
Greedy attack (our work)	PoA	VeChain	$< \frac{N}{3}$	○	Finality cannot be achieved; finalization prob. $\approx 0.19\%$

● = requires attacker-controlled network actions (e.g., partitioning, selective delivery, or strong propagation control); ○ = does not require attacker network control.

sufficiently large fraction of sealers and create a fork. As a result, safety is violated. The assumption, however, might be overly strong. Wang et al. [43] proposed two types of *front-running* attacks where the adversary could manipulate the order of the transactions to gain additional benefits. Zhang et al. [50] presented *time manipulation* attack [47] to the PoA Aura protocol. We summarise more attacks in Table 1. Our work focuses on the finality of VeChainThor, different from all prior lines.

C Additional Analysis on POA Aura

In Aura, finality is achieved once a block B has been extended by blocks proposed by at least $N/2$ distinct sealers. Under our greedy attack, the probability that subsequent blocks are proposed by honest sealers decreases significantly. Consequently, even Aura, which does not rely on a dedicated finality gadget, exhibits delayed confirmation.

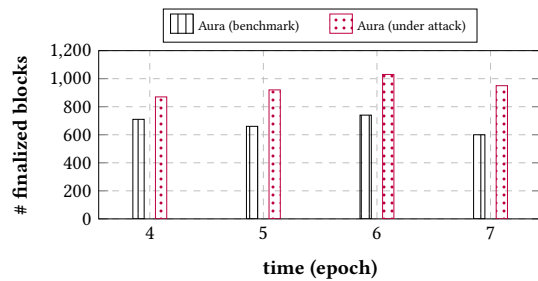


Figure 10: Number of finalized blocks in Aura with and without the greedy attack. Each epoch contains two bars: the first bar shows the benchmark and the second bar shows the result under attack.

In the benchmark (no attack), the number of blocks finalized per epoch is roughly the same (on average 678 blocks, where the number of blocks in epochs 4–7 is: 710, 660, 740, 600, respectively).

Under the greedy attack, this per-epoch finalization rate declines to an average of 943 blocks (the number of blocks in epochs 4–7 is: 870, 920, 1030, 950, respectively).

D Ethics Considerations and Open Science Policy Compliance

In this section, we show our commitment to ethical research practices and open science principles. We declare that all our experiments do not involve any issues related to animals, human beings, the environment, healthcare, or military factors. We have addressed numerous ethical considerations in our experimental design, strictly adhering to the ethical principles. In our work, we adhered to the following ethical principles:

- **Attack disclosure:** We responsibly reported our attack findings to the VeChain team in April 2024, more than half a year ago. All discussions regarding this attack will be made publicly available to the community upon publication.
- **Live system testing:** We have not conducted experiments on the live VeChain mainnet. All experiments were performed using the controlled local testnet.
- **Beneficence:** Our research aims to improve the security and stability of the VeChain network, providing a net positive benefit to the community. We have carefully weighed potential negative outcomes against these benefits. Our findings can help raise the community's security awareness.
- **Justice:** We strive to ensure our proposed modifications do not disproportionately impact or disadvantage any particular group of VeChain users and validators.
- **Respect for persons:** Our research does not involve human subjects or personal data. We respect the work of other researchers and properly cite all relevant prior work.
- **Respect for law and public interest:** Our research complies with all applicable laws and regulations. We have considered the broader societal implications of more secure blockchain systems.